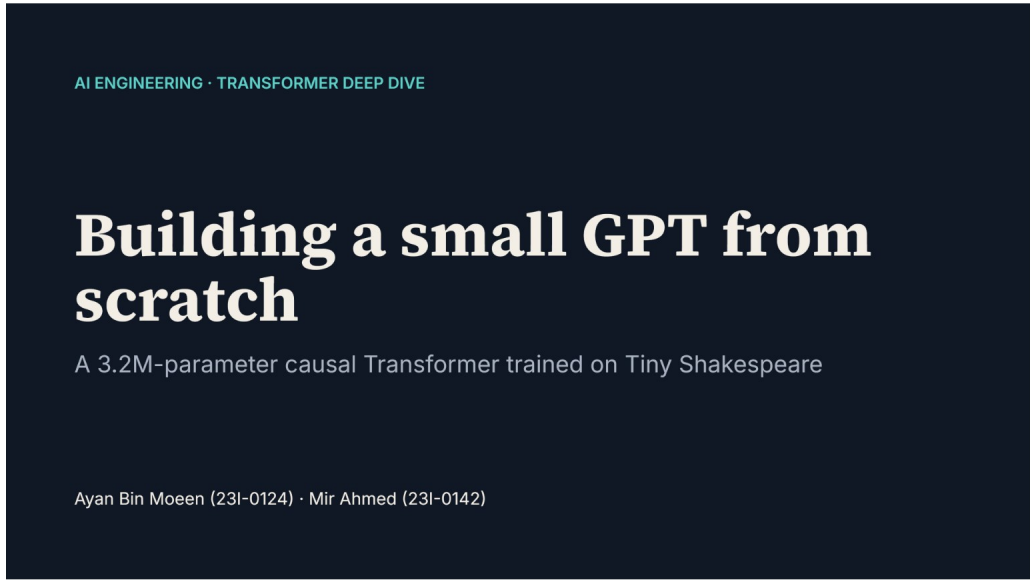
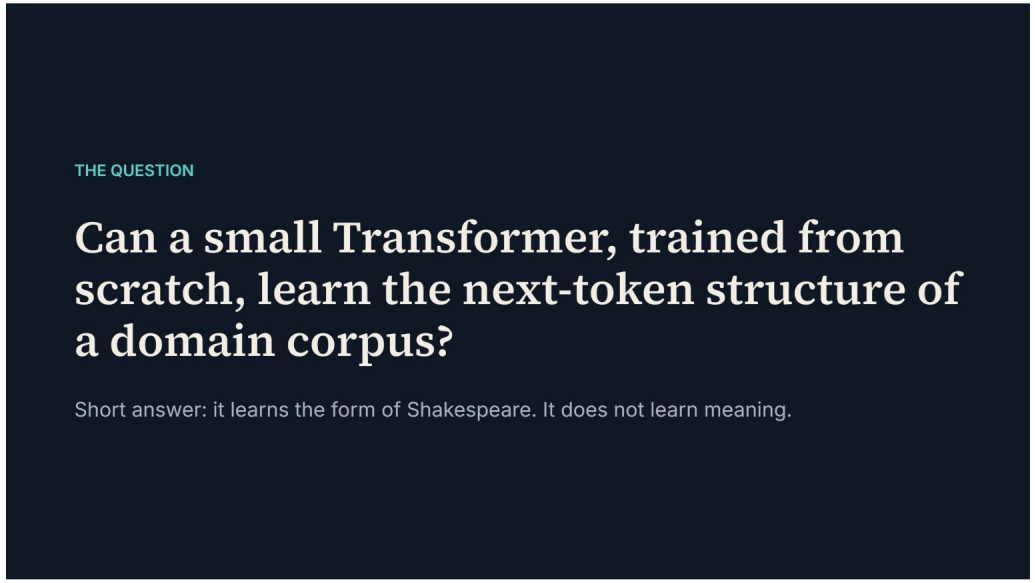
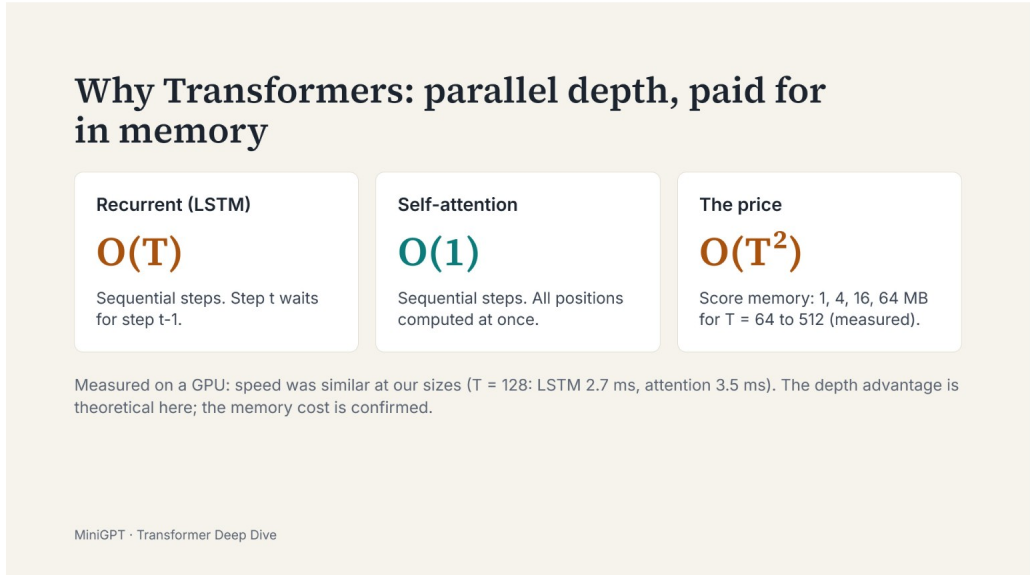


Introduce yourselves and the project. One sentence: we built a miniature GPT where every Transformer component is our own code, trained it on Shakespeare, tested it, ran ablations, and deployed it as an API.

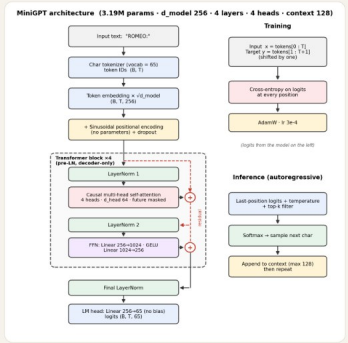


Scenario: a small team with a private corpus too small for a foundation model. Our lead engineer asks whether a compact Transformer can still learn the next-token structure. We answer with a model we built ourselves and with measurements. Give the short answer now and promise the evidence.



Lesson 1. An LSTM must process positions one after another, so its sequential depth grows with sequence length. Attention connects all positions in one operation, so depth is constant, but it stores a T by T score matrix per head. We measured both. Be honest: on a small GPU workload the LSTM was just as fast, so we report the depth advantage as theory and the memory growth as measured.

The MiniGPT architecture



3,192,832
trainable parameters

d_model 256 · 4 heads · 4 layers
FFN 1024 with GELU, dropout 0.1

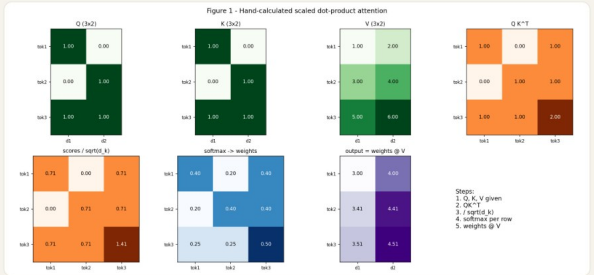
Context 128 · vocab 65 characters
Character-level tokenizer, no special tokens

Pre-LayerNorm + residuals
Sinusoidal positions, untied LM head

MiniGPT · Transformer Deep Dive

Walk the diagram top to bottom: characters become IDs, embeddings plus sinusoidal positions, four pre-LayerNorm blocks (causal multi-head attention and a GELU feed-forward network, each with a residual), a final LayerNorm and a linear head to 65 logits. Parameter count: 16,640 embedding, 789,760 per block, 512 final LayerNorm, 16,640 head. Everything here is our own code; PyTorch only supplies tensors, linear layers, LayerNorm and autograd.

Attention by hand: 3 tokens, d_k = 2

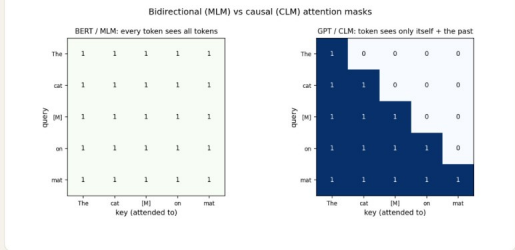


With the causal mask, token 1 sees only itself: its output becomes exactly [1, 2]. Token 2 mixes two tokens ([3.41, 4.41] becomes [2.34, 3.34]). Token 3 already sees everything, so it does not change.

MiniGPT · Transformer Deep Dive

This is the required hand calculation. Q equals K equals [[1,0],[0,1],[1,1]], V is [[1,2],[3,4],[5,6]]. Compute QK transpose, divide by root 2, softmax each row, multiply by V. Then add the causal mask: forbidden scores become minus infinity, so their weights are exactly zero and each row still sums to one. Explain what changes: row 1 can only copy its own value, row 2 mixes two tokens, the last row is unchanged.

Why GPT is causal and BERT is not



Experiment: change only the last token, then look at the output at position 0.

0.127

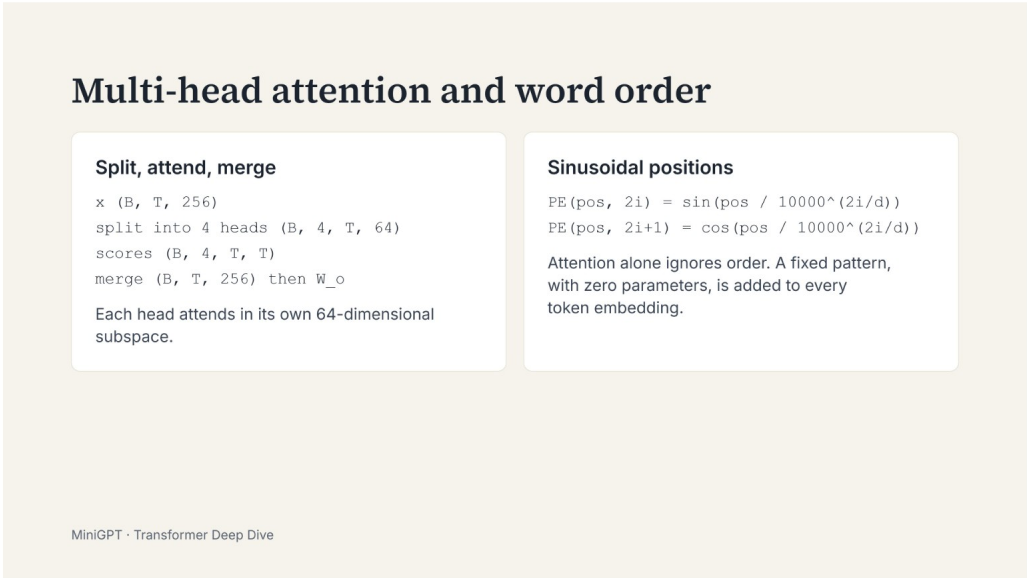
Bidirectional: position 0 sees the future

0.000

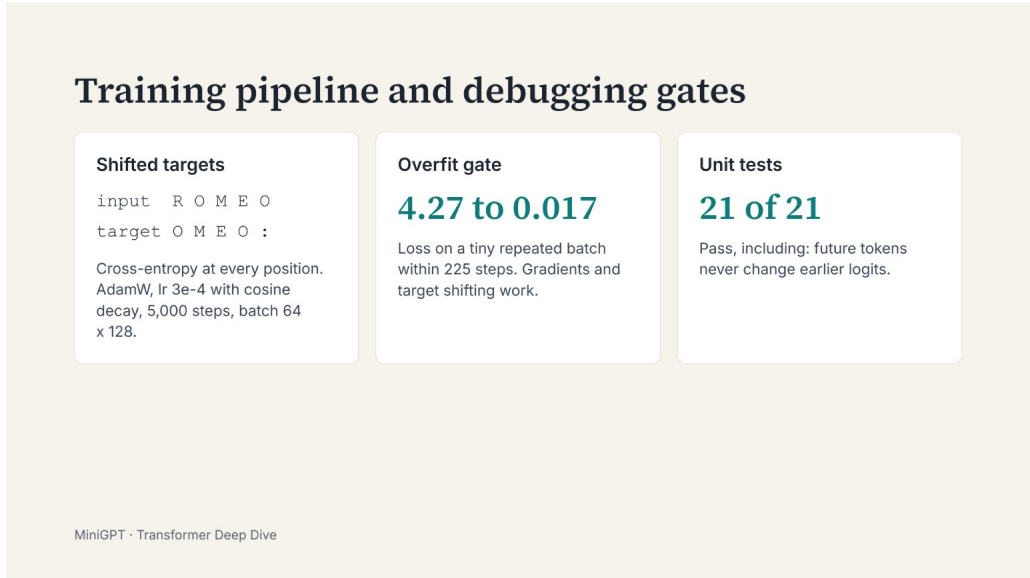
Causal: the future is blocked

MiniGPT · Transformer Deep Dive

BERT hides some tokens and predicts them from both sides, so its attention matrix is full. GPT predicts the next token; if a position could see the future it would just copy the answer, so the matrix must be lower-triangular in training and in generation. Our comparison: changing the last token moves position 0 by 0.127 in the bidirectional case and by exactly zero in the causal case. The same property is a unit test on the full model.

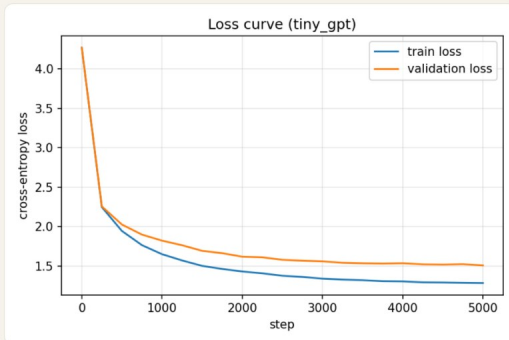


Multi-head: project to Q, K, V, split the 256 dimensions into 4 heads of 64, run attention in each head, concatenate, and apply an output projection. Split and merge are explicit reshapes in our code, and the shape trace is a figure in the report. Positions: attention treats the sequence as a set, so we add a sinusoidal encoding; low dimensions oscillate fast, high dimensions slowly, so each position gets a unique signature.



The input is characters 0 to T-1 and the target is characters 1 to T, so the target at position t is the next character after the input at position t. Before the full run we overfit a tiny repeated batch: loss fell from 4.27 to 0.017 in 225 steps, which proves gradients and target shifting are correct. The test suite has 21 tests in 5 files. Show the pytest screenshot in the demo if asked.

Learning curve: train 1.28, validation 1.51



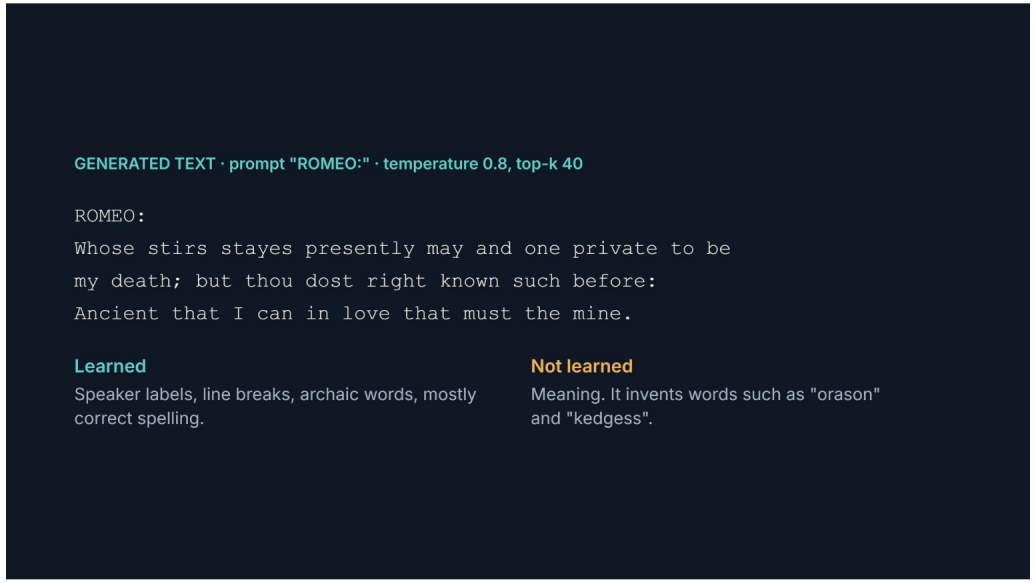
4.51
Validation perplexity, out of 65 characters

7.6 min
5,000 steps on a Colab T4 (455 s)

0.22
Train-validation gap after about 40 passes over 1 MB

MiniGPT · Transformer Deep Dive

Loss starts near 4.27, close to uniform guessing over 65 characters (ln 65 is about 4.17), and falls fast in the first 500 steps. It ends at 1.28 on training data and 1.51 on validation data, perplexity 4.51. The gap widens slowly but validation loss never rises, so overfitting is mild; more data would help more than more steps. Mention the validation loss was monitored during training, not only at the end.



Five fixed prompts are in the report; this is the first. The model clearly learned the format of a play and Shakespearean vocabulary, but sentences do not make sense and it occasionally invents words. Interpretation: a character-level model with 3M parameters and 1 MB of data learns spelling, local grammar and layout, not meaning. Be ready to run one live generation here.

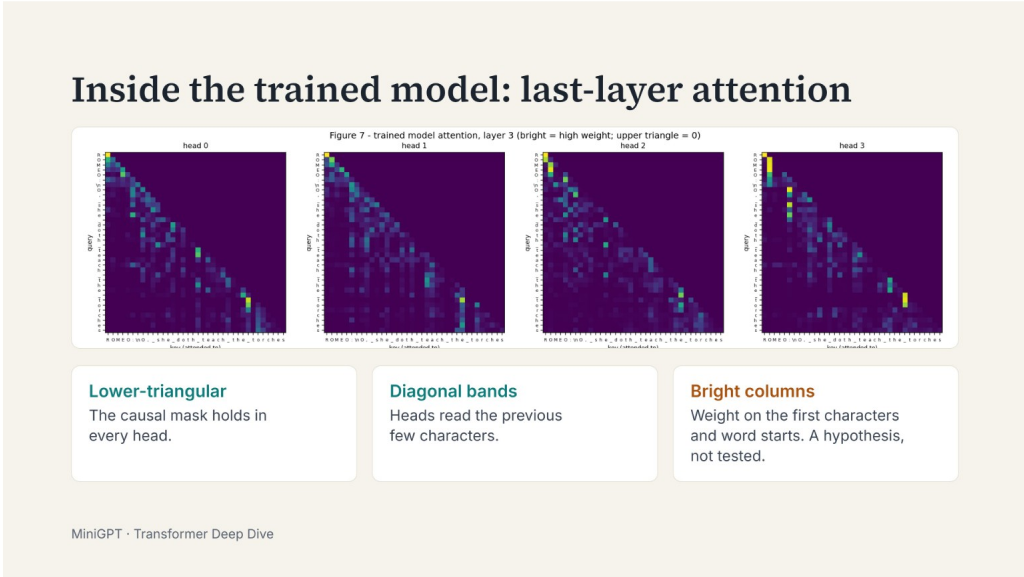
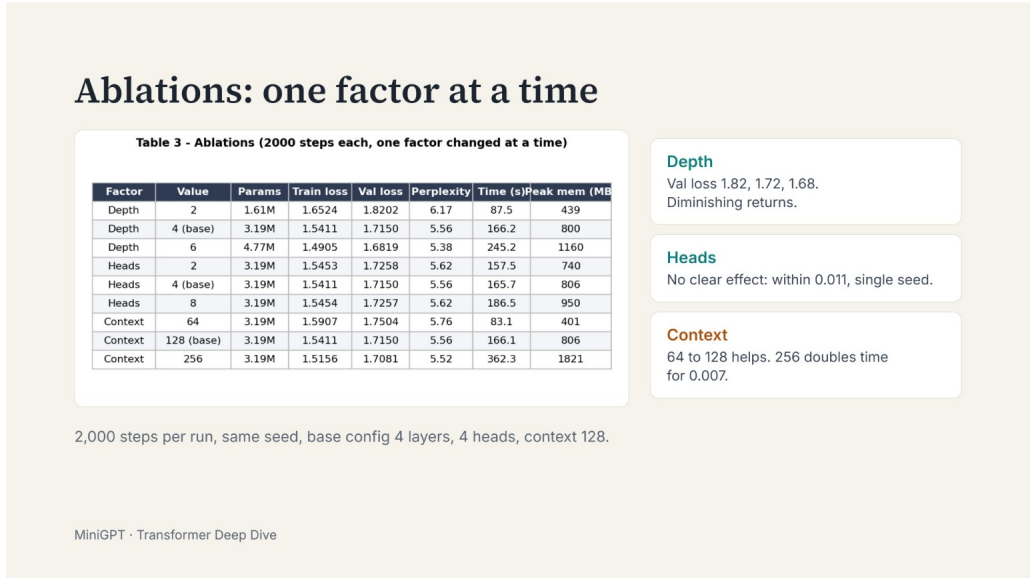
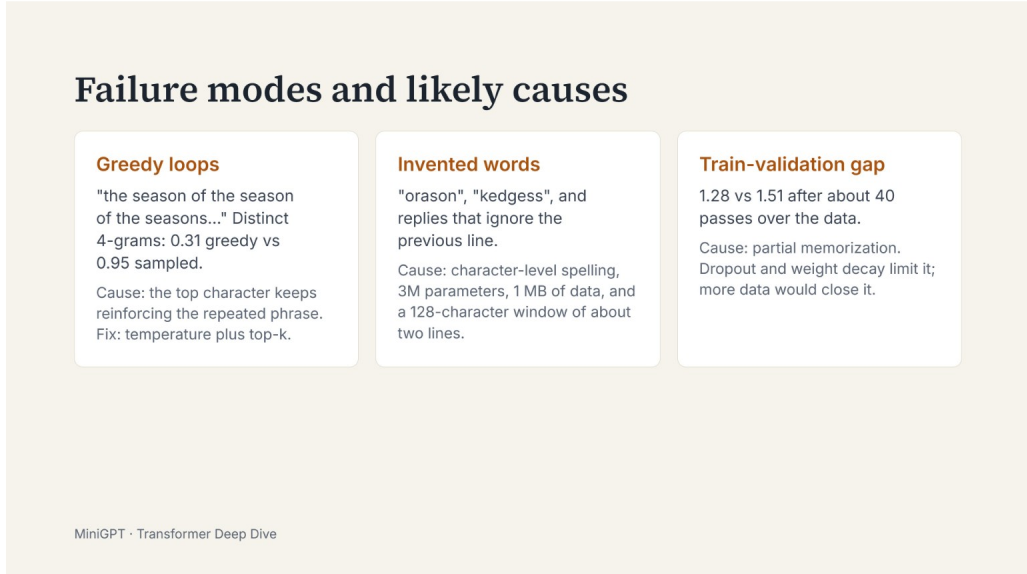


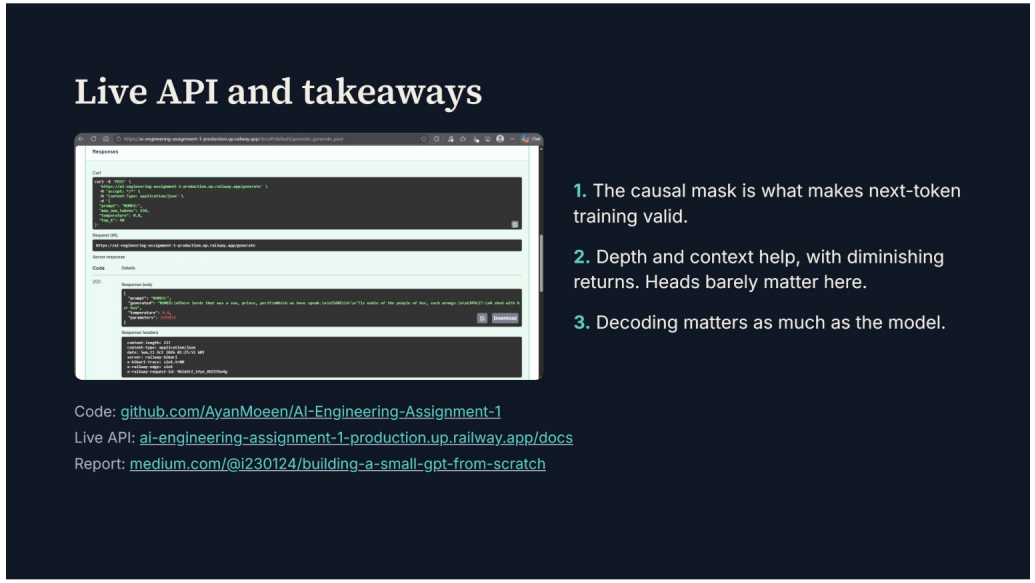
Figure 7: attention weights of the last layer on the prompt "ROMEO: O, she doth teach the torches". Observation: every head is strictly lower-triangular, most put weight near the diagonal, and several show bright vertical stripes on the first characters and on word-initial letters. Interpretation: heads seem to combine local spelling context with a few longer lookups. We did not run causal interventions, so say this is a hypothesis.



Three controlled experiments, each changing one factor. Depth: validation loss 1.8202, 1.7150, 1.6819 for 2, 4, 6 layers; the first doubling gains 0.105, the next only 0.033, and time grows about linearly. Heads: 1.7258, 1.7150, 1.7257 with identical parameter count; the difference is small and we only ran one seed, so we do not claim an effect. Context: 1.7504, 1.7150, 1.7081; time 83, 166, 362 seconds and memory 401, 806, 1821 MB, consistent with quadratic attention memory. Caveat: batch size is fixed, so longer context also means more tokens per step. Ablations used 2,000 steps, so their base loss is higher than the final 5,000-step model.



The handout asks for at least two failure modes; we show three. Greedy decoding collapses into a repetition loop. Samples contain invented words and weak coherence because the model is character-level, small, and trained on little data with a short window. The training-validation gap shows partial memorization. For each one, state the observation first, then the likely cause as an interpretation.



Switch to the live API at /docs and run one POST /generate call with prompt ROMEO: to show the deployed model. The screenshot is the backup if the service is asleep or the trial credit is used up. Then close with the three takeaways and name one limitation: the model learns the form of Shakespeare, not its meaning. Invite questions; both of us should be ready to explain any component.